Computer Vision

Assignment 2 Report:

Ahmad Yasser Mohammad Salaheldin Hamad (ID:9074)

Moamen Talaat (ID:9103)

Part 1 (Augmented Reality):

The objective of this section is to be able to develop an augmented reality (AR) video, using two source videos. The first video shows two books: Computer Vision (A Modern Approach) and Multiple View Geometry (in computer vision). The second video is the trailer of the animated movie: Kung Fu Panda. Accordingly, the aim of this task is to be able to overlay every frame of the Kung Fu Panda movie over the Computer Vision book (in particular, not the Multiple View Geometry book) in the second movie.

Code Explanation:

For simplicity, the code begins according to the first frame of both the animated movie and the video showing both books. Once done and completed, it can be extended across all frames in a simple pipeline that can easily streamline the entire process.

To begin with the overlay procedure, the location of the book in the video must be clearly identified. Knowing that, we use the SIFT (scale invariant feature transform) descriptor. This descriptor returns keypoints and descriptors for each image. To properly match the book from the video, a separate, clearer cover of the book is provided. For both images (the first frame of the video and the book's cover), the SIFT descriptor is applied. The outcome is shown below:

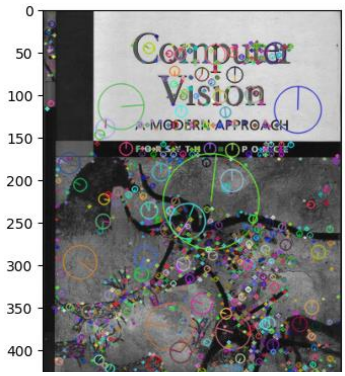

Figure 1 SIFT outcome on book cover

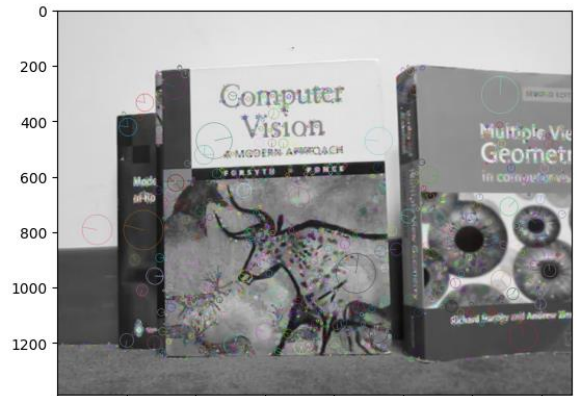

Figure 2 SIFT outcome on first frame of the video

The SIFT descriptor was able to successfully find blobs in both images and returned the keypoints and descriptors found for each image.

Moving forward, a brute force mechanism is employed to match descriptors from figures 1 and 2. The matcher used was a K-Nearest-Neighbor (KNN) matcher, having a K of 2. Therefore, for every descriptor, we are returning two matches, the nearest two neighbors. According to Lowe's paper, a ratio of 0.65 is appropriate enough to determine if a match is suitable or not. Given that we are returning two neighbors, if the distance to the first (nearest) neighbor over the distance of the second neighbor is greater than or equal to 0.65, the match is considered ambiguous and is therefore not considered as a good match. If it is less than 0.65, the match is deemed a good match. Better (typically more confident) matches would have lower distances. Accordingly, after filtering out the good matches from the bad ones, the good matches are then ordered by their ratios, in which the lower ratio would be first. This would leave the more confident matches in the beginning of the matches' array. Moreover, the first 50 matches are then taken (not all matches) for the sake of robustness and accuracy. Plotting the matches between figure 1 and 2 yields the following:

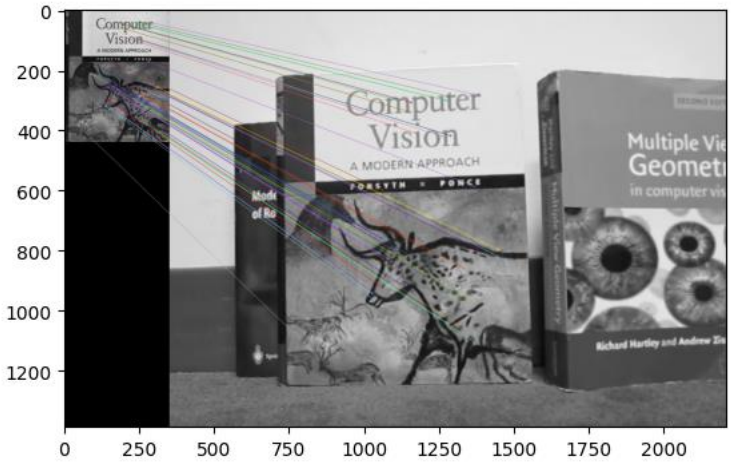

Figure 3 Matches between figure 1 and figure 2

Given the matches between the first and the second figures, the goal now is to compute the homography matrix that transformed points in figure one to figure 2. To do so, the following procedure is employed:

Finding the homography matrix through solving a system of equations. Each correspondence (x, y) provides 2 equations, one for the x variable and one for the y variable. 8 unknowns in H are stacked on top of one another, and the 9th term (3,3) is set to 1. To find the 8 unknowns, we need at least 4 correspondences. We will therefore attempt to solve Ah = 0. h contains all 8 unknown homography parameters stacked on top of each other. A is a 2n * 9 matrix, n being the number of correspondences. The Singular Value decomposition (SVD) algorithm is employed on the homography matrix, and the stored resultant vector is the one related to that of the smallest singular value. It is then normalized by dividing the final 9th term across all values in the matrix. To determine the validity of the resultant homography matrix visually, 10 random points from figure 1 are mapped to figure 2. If the correspondences are correct, then the resultant matrix is a correct one.

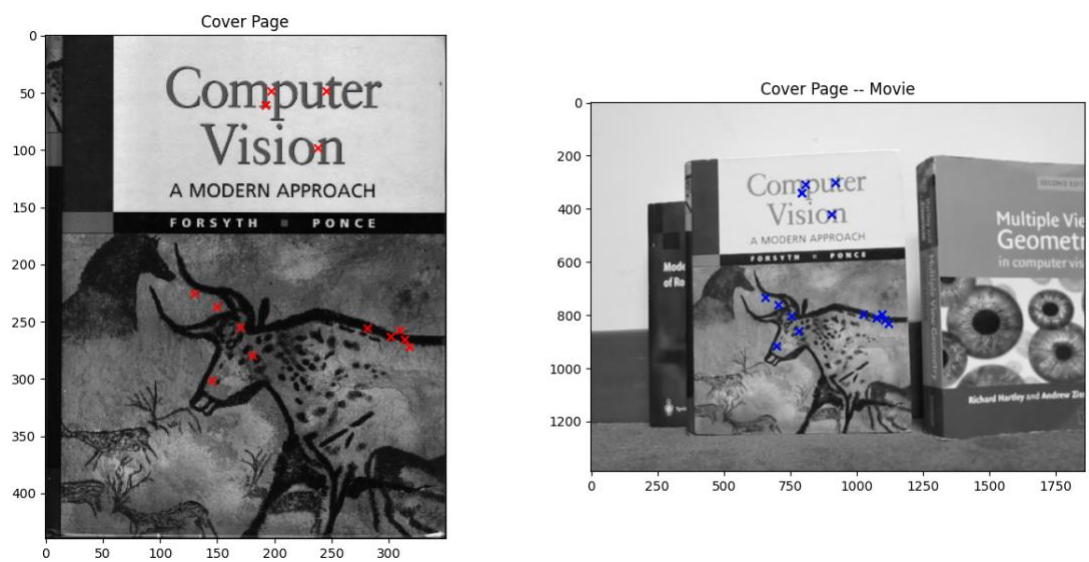

Figure 4 10 correspondences between figure 1 and figure 2 plotted

To determine the boundaries of the book in preparation for overlaying the first frame of the movie over it, the book cover's corners are transformed using the matrix to match those of the first frame of the video containing the textbook. The resultant corners are correctly defined.

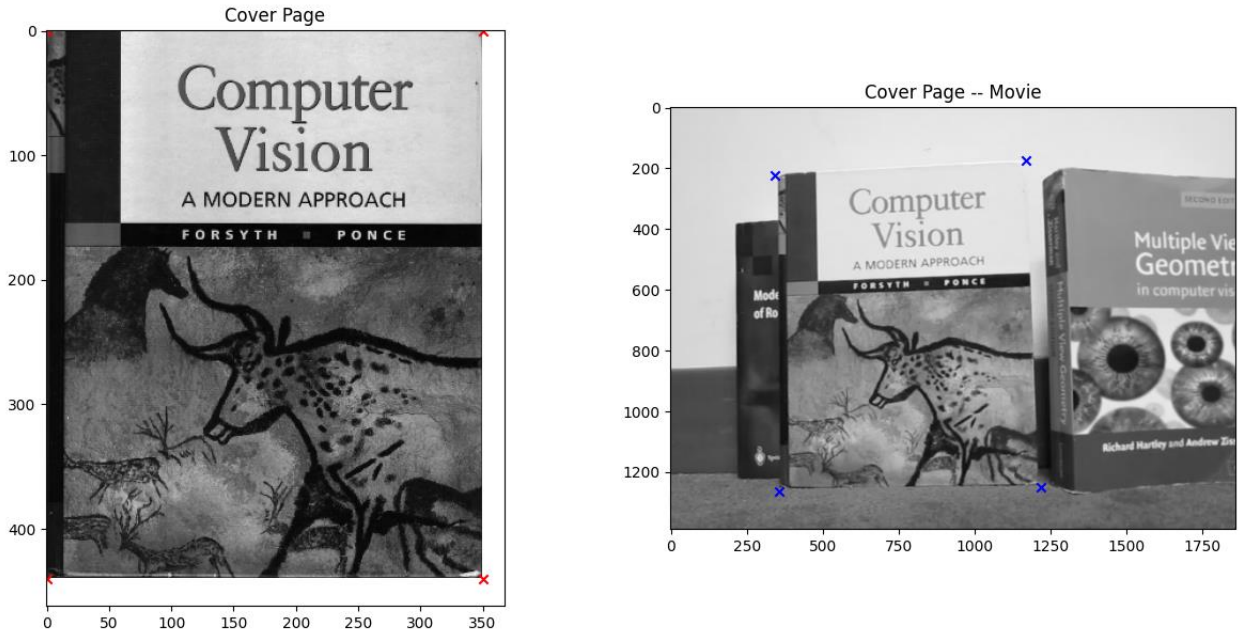

Figure 5 Mapping the book's corners from figure 1 to figure 2

Moving forward to the movie's first frame, the entire movie frame cannot fit into the textbook and would accordingly have to be cropped, maintaining the center of the current movie frame in the center of the book. The following calculation was done:

The dimensions of the movie are 640 (width) x 360 (height), while the dimensions of the book's cover 350 x 440. Therefore, at the center of the movie frame (320), we will keep everything at a margin of 175 pixels. Therefore the leftmost value will be at 145 and the rightmost value will be at 495. Unfortunately, this will result in incorrect aspect ratios (image width/ image height), as required. The book cover's aspect ratio is 350/440 = 0.795. Accordingly, after cropping the first frame of the movie, we must maintain a similar aspect ratio (currently, it is 350/360 = 0.972222). Given that the movie's height will not change, the only available variable

is the width size. Therefore the width will evaluate to approximately 286 pixels. Accordingly, at the center of the movie frame (320) we will keep everything at a margin of 143 pixels. Therefore, the leftmost value will be at 177 and the rightmost value will be at 463. Considering the movie's first frame:

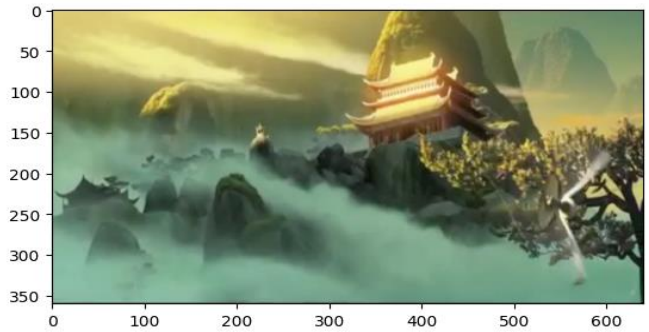

Figure 6 First frame of the animated movie

Figure 7 First frame of the animated movie after cropping

A second obstacle that had to be dealt with is the difference in sizes between the movie's frame size and the book cover's size. This was simply resolved by a built-in resize function that changes the size of an input image to the target size needed.

To properly overlay the first frame, the following was done. The frame was transformed using the obtained homography matrix. Next, the background (the image containing the first frame of the book) was kept as is, with the exception of coloring the corners of the book (previously obatined from an earlier step) white. This was then inveresed, keeping everything white except the position in which the book should be placed. A bitwise_and between the original background and the masked background was done, leaving the

Figure 8 Movie frame after processing

entire background present with a black cutout of the book in it. The transformed frame is inserted into the black cutout, providing the overlay effect as follows:

The rest of the video is obtained in the same exact manner, as each video is processed separately, with the frame currently being viewed being provided by predefined functions. The homography matrix is recreated in each and every frame and compared with the book's cover.

Figure 9 Overlaying the movie's first frame over the book

Part 2 (Image Stitching):

The objective

The goal of this project was to develop a robust system for stitching multiple images into a seamless panorama. This involved detecting matching features between images, calculating Homography, and warping images into a unified coordinate system using Per Channel Inverse Warping and Weighted Blending.

A. Feature Detection and Matching

We used the SIFT algorithm to extract keypoints and descriptors.

Matching: We used a Brute-Force Matcher with KNN (k=2).

- Quality Control: We applied Lowe's Ratio Test. By ensuring the distance of the best match was less than 75% of the second-best match

B. Homography Estimation

Using the filtered correspondences, we computed a 3x3 Homography matrix (H).

Direct Linear Transformation (DLT): We set up a system of linear equations (Ax =b) for 8 unknowns, fixing H(3,3) = 1.

Least Squares Optimization: Since we used ~50 points, we solved the overdetermined system using np.linalg.lstsq to minimize the geometric error and produce a stable alignment.

C. The Unified Coordinate System

To prevent images from being cut off at negative coordinates:

1. Corner Projection: We projected the corners of the warped image to find the global boundaries
2. Global Translation (T): We created a translation matrix to shift the entire scene so the top-left corner sits at (0,0).
3. Warping:
    - The Warped Image: Transformed by the combined matrix M = TxH.
    - The Anchor Image: Shifted only by the translation T to maintain its original perspective.

D. Per-Channel Backward Warping

To achieve the highest visual quality we implemented a manual Inverse Warp logic:

Split-Warp-Merge: The BGR image was split into 3 independent 2D channels. Each was warped separately and then merged.

Bilinear Interpolation: During the inverse lookup, we calculated a weighted average of the 4 surrounding pixels.

E. Image Blending

To hide the line between images and the differences of the intensity , we implemented Weighted Blending: I= (W1I1+W2I2)/(W1+W2)

Figure 10 Results of the Mosaicing using two photos

Figure 11 Results of the mosaicing using three photos